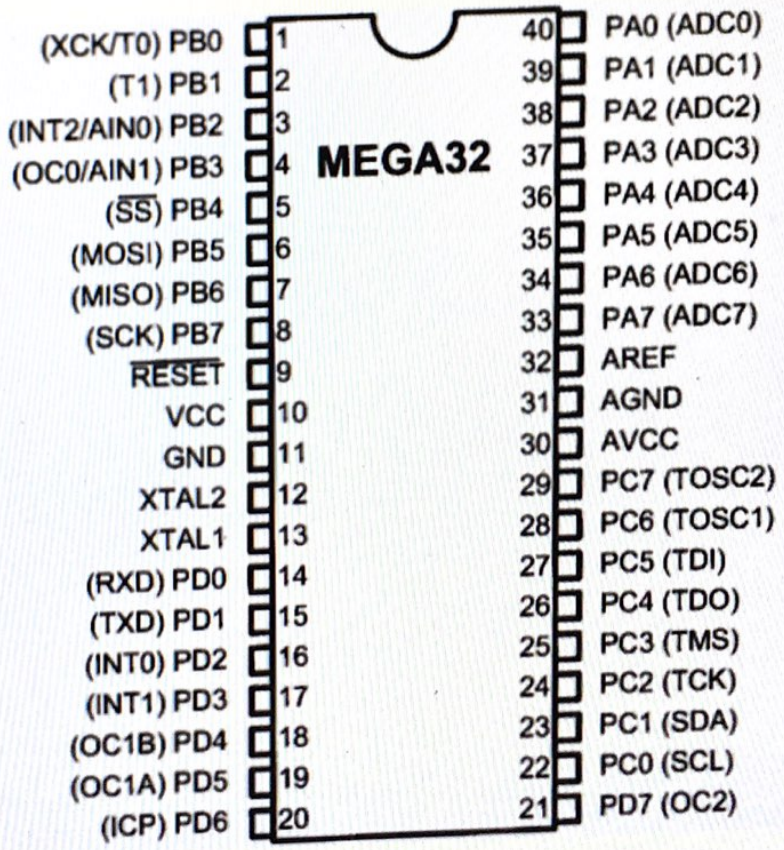
# Chapter 4-Fig4-1



USART (Universal Synchronous/Asynchronous Receiver/Transmitter in the AVR

4 features:
- normal asynchronous
- double-speed asynchronous
- master synchronous
- slave synchronous

Synchronous
- ADC
- EEPROM communication

asynchronous
- COM PORT x86
- full-duplex capability

AVR registers associated with USART
- UDR (USART Data Register)
- control registers: UCSRA, UCSRB, UCSRC
- UBRR (USART Baud Rate Register)

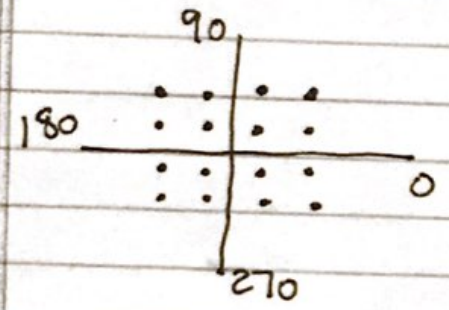
band rate - Symbols per second

bits per second (bps)

Ex) cable signal

QAM (quadrature amplitude modulation)

QAM-16




phase, magnitude

$2^4$ (3 data bits + 1 error bit)

---

overhead

COM Port IBM & Compatibles { 8 bits, Start bit, Stop bit, parity bit

9600

% overhead

Start, stop, parity

$\frac{1+1+1}{8+1+1+1} = \frac{3}{11} \Rightarrow 27\%$

Start, Stop

$\frac{1+1}{8+1+1} = \frac{2}{10} \Rightarrow 20\%$

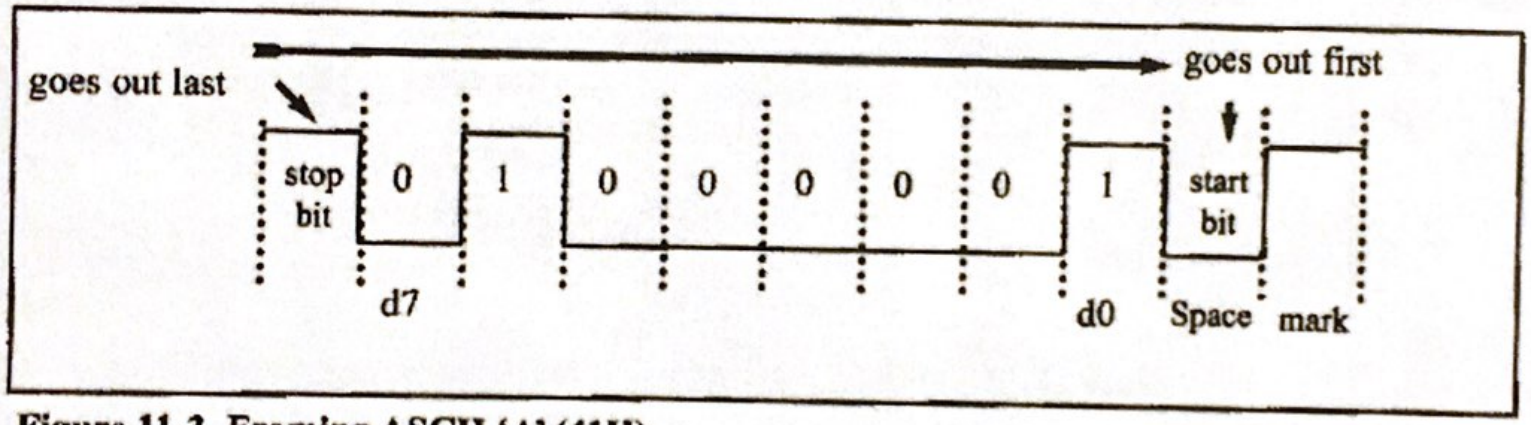
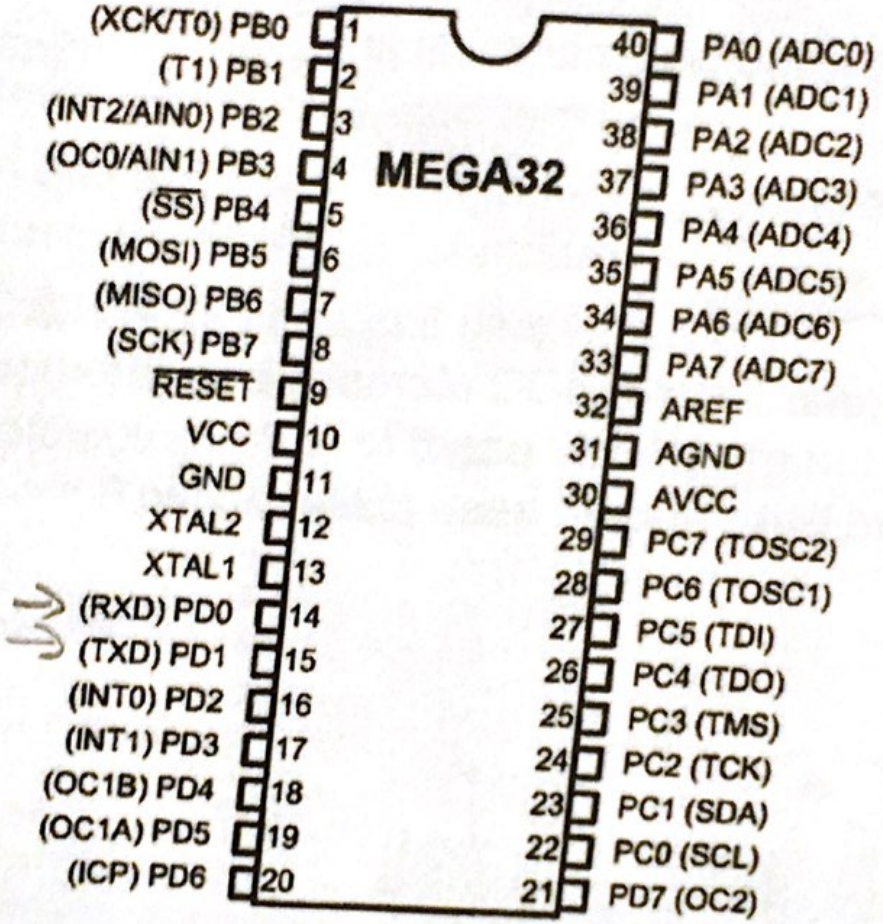


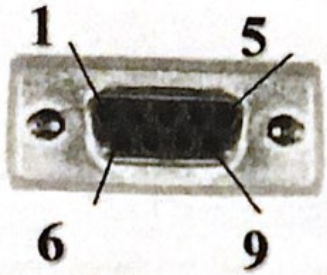

**Figure 11-3. Framing ASCII 'A' (41H)**

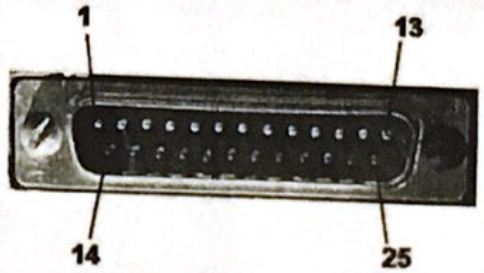
# Chapter 4-Fig4-1



# RS-232C

- TTL logic signals are not what is transmitted between equipment such as modems, computers, and terminals.
- One standard that specifies the signals for asynchronous communication is the RS-232C standard.
- The RS-232C specifies 25 signal pins with a male DTE (Data Terminal Equipment) and a female DCE (Data Communications Equipment) connector. Most of these signals are not used in most applications, so a 9 pin subset is used (popularized by the IBM PC or compatible).



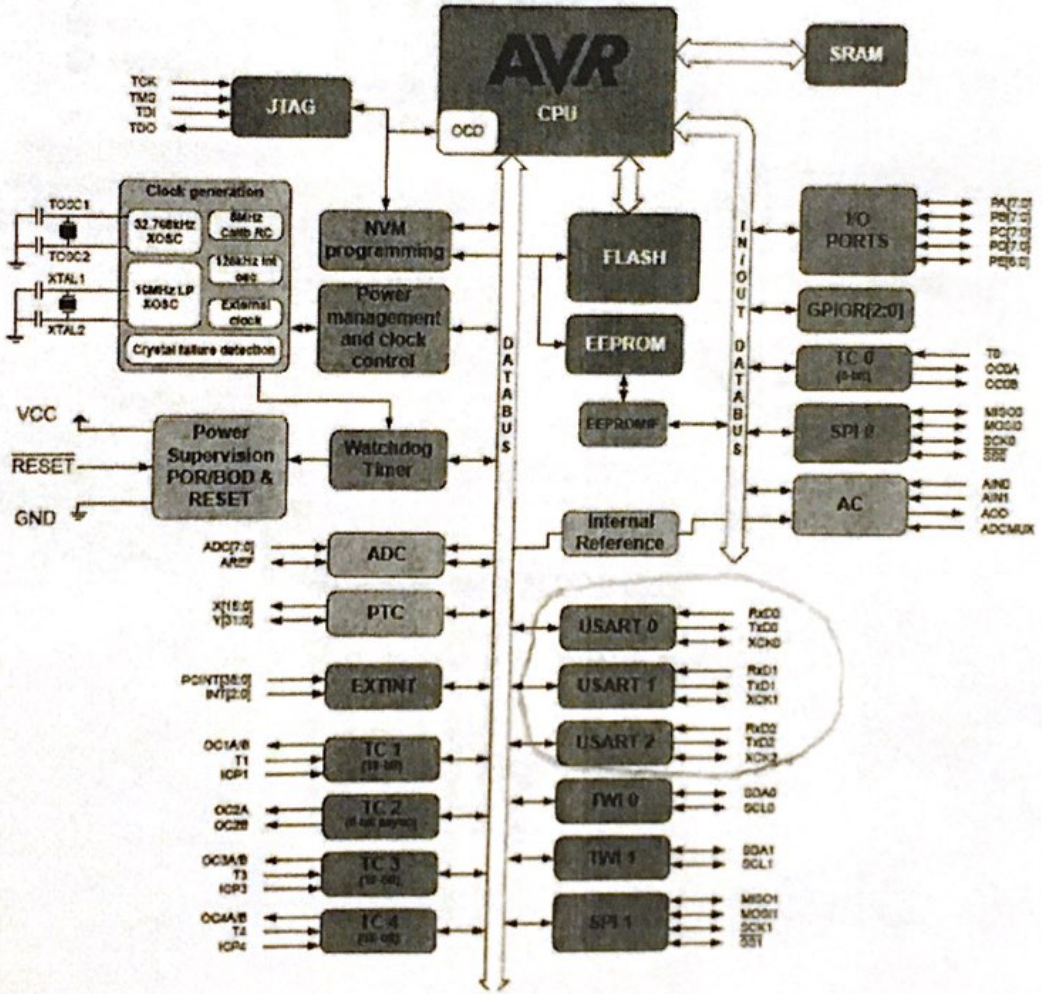
# Important Asynchronous Signals in RS-232C

| Description | PIN # (9-PIN) | From | Abbreviation |
|---|---|---|---|
| **Data Leads** | | | |
| Transmit Data | 3 | DTE | TD |
| Receive Data | 2 | DCE | RD |
| **Power On Indicator Leads** | | | |
| Data set ready | 6 | DCE | DSR |
| Data terminal ready | 4 | DTE | DTR |
| **Leads that announce that an outside event has taken place** | | | |
| Data carrier detect | 1 | DCE | CD |
| Ring Indicator | 9 | DCE | RI |
| **Ready to send/receive handshake leads** | | | |
| Request to send | 7 | DTE | RTS |
| Clear to send | 8 | DCE | CTS |
| **Ground leads** | | | |
| Signal ground | 5 | | SG |

# ATmega324pb

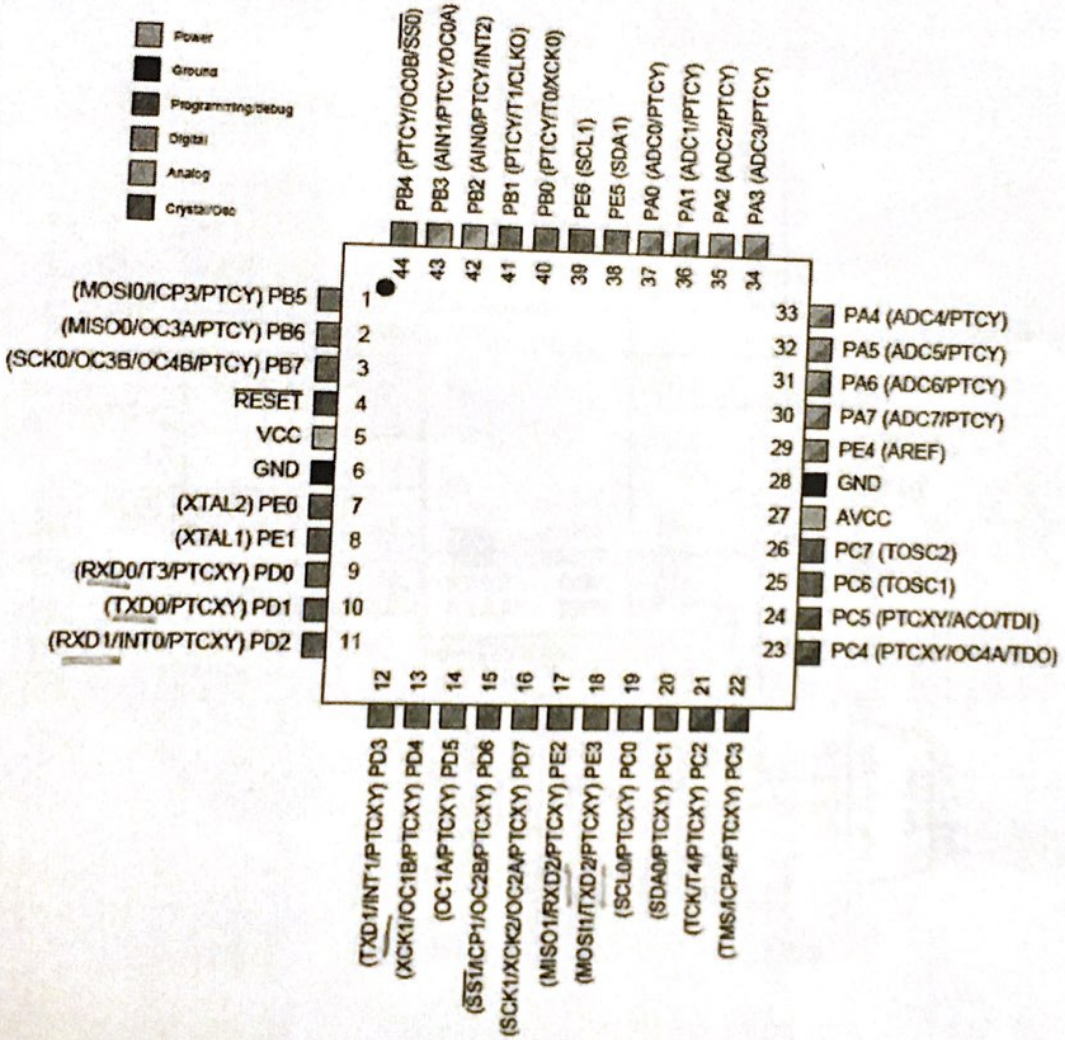
## Block Diagram

**Figure 4-1. Block Diagram**

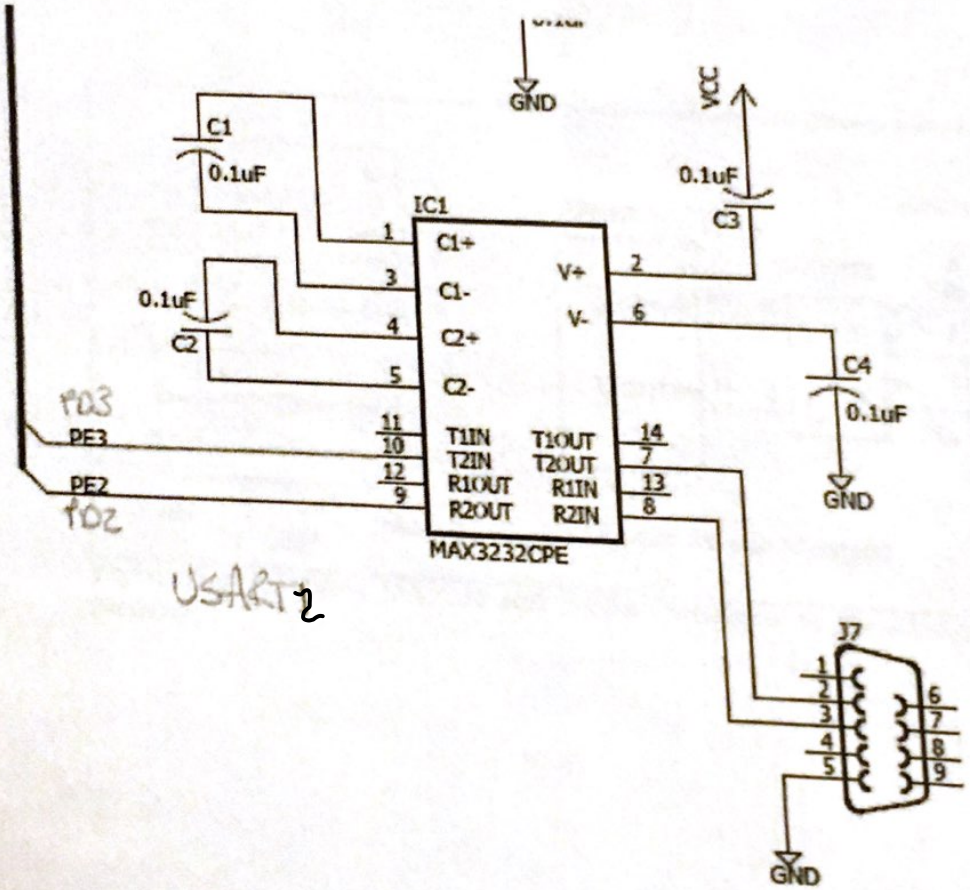


# ATmega324pb

**Figure 5-1. Pinout ATmega324PB**



# ATmega324pb



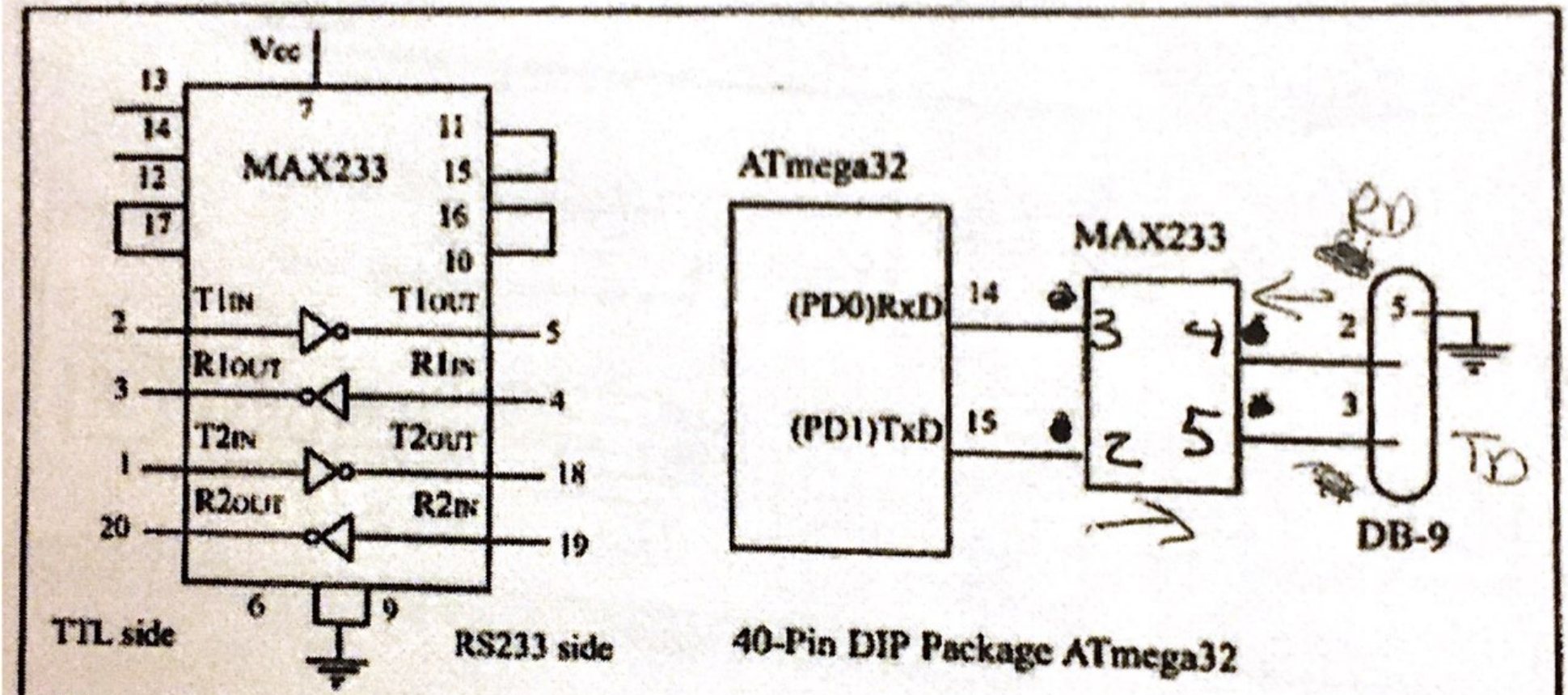



**Figure 11-8. (a) Inside MAX233 and (b) Its Connection to the ATmega32 (Null Modem)**

UDRE

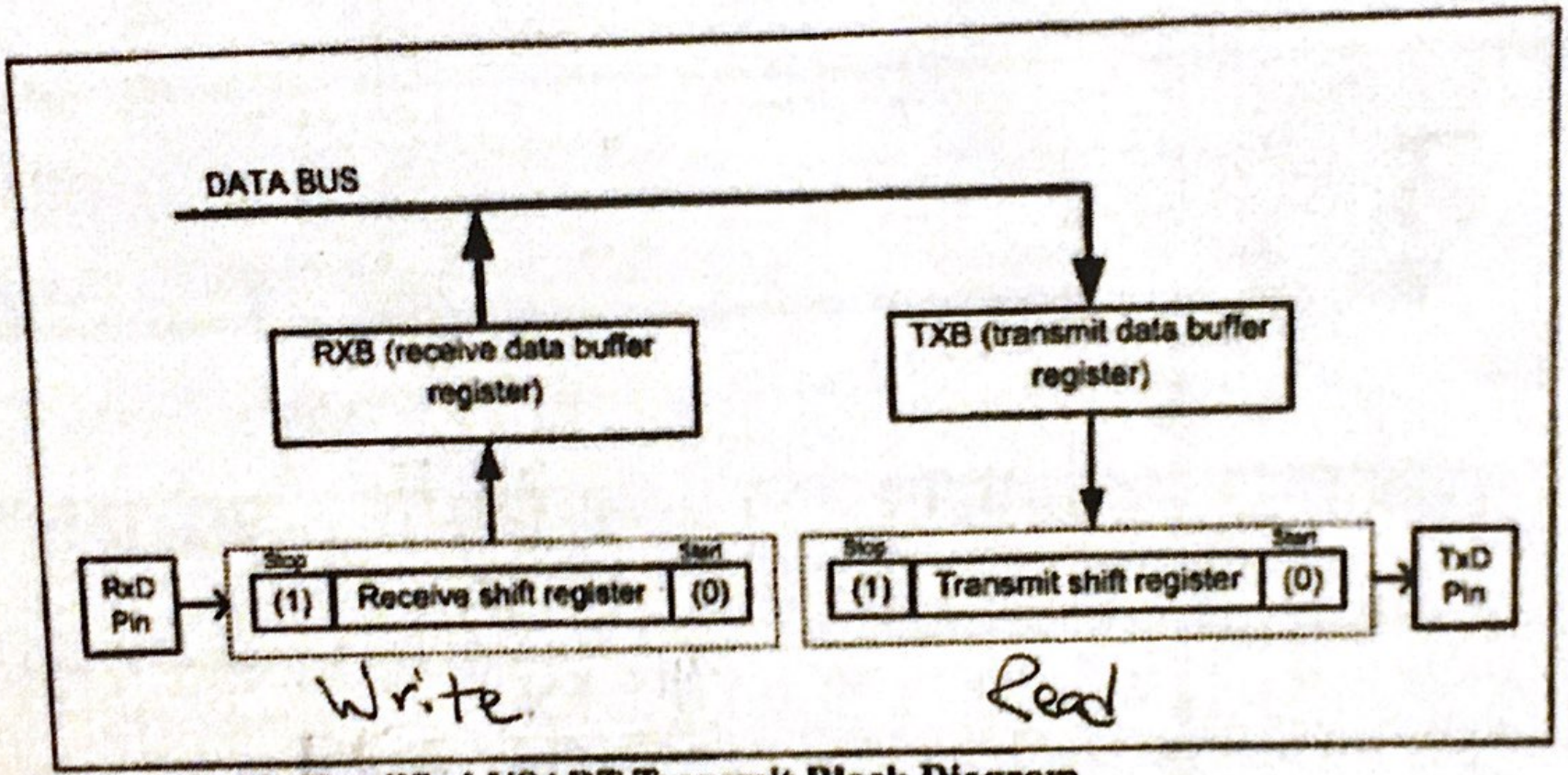



**Figure 11-12. Simplified USART Transmit Block Diagram**

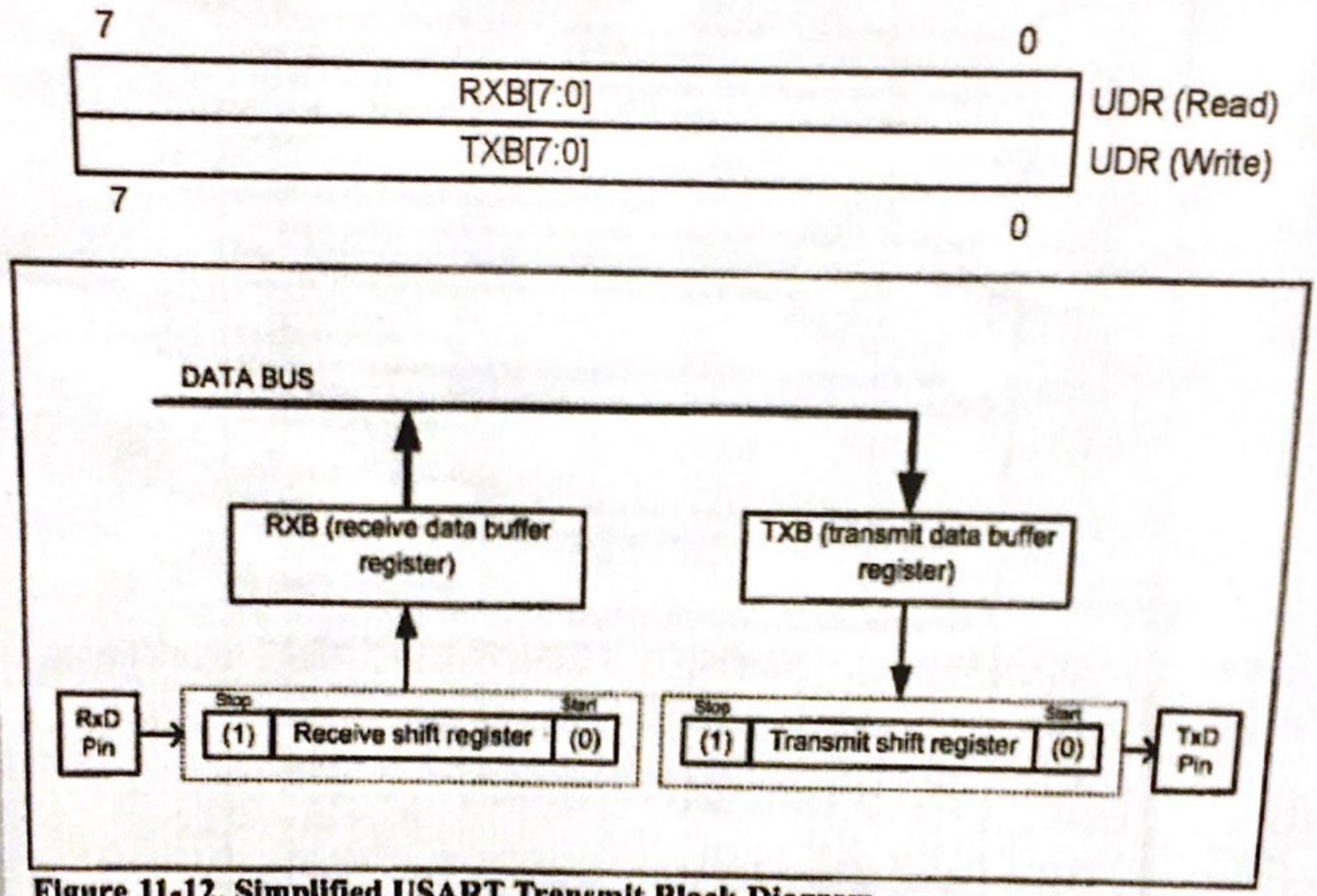



**Figure 11-12. Simplified USART Transmit Block Diagram**

| RXC | TXC | UDRE | FE | DOR | PE | U2X | MPCM |
|---|---|---|---|---|---|---|---|

**RXC (Bit 7): USART Receive Complete**
This flag bit is set when there are new data in the receive buffer that are not read yet. It is cleared when the receive buffer is empty. It also can be used to generate a receive complete interrupt.

**TXC (Bit 6): USART Transmit Complete**
This flag bit is set when the entire frame in the transmit shift register has been transmitted and there are no new data available in the transmit data buffer register (TXB). It can be cleared by writing a one to its bit location. Also it is automatically cleared when a transmit complete interrupt is executed. It can be used to generate a transmit complete interrupt.

**UDRE (Bit 5): USART Data Register Empty**
This flag is set when the transmit data buffer is empty and it is ready to receive new data. If this bit is cleared you should not write to UDR because it overrides your last data. The UDRE flag can generate a data register empty interrupt.

**FE (Bit 4): Frame Error**
This bit is set if a frame error has occurred in receiving the next character in the receive buffer. A frame error is detected when the first stop bit of the next character in the receive buffer is zero.

**DOR (Bit 3): Data OverRun**
This bit is set if a data overrun is detected. A data overrun occurs when the receive data buffer and receive shift register are full, and a new start bit is detected.

**PE (Bit 2): Parity Error**
This bit is set if parity checking was enabled (UPM1 = 1) and the next character in the receive buffer had a parity error when received.

**U2X (Bit 1): Double the USART Transmission Speed**
Setting this bit will double the transfer rate for asynchronous communication.

**MPCM (Bit 0): Multi-processor Communication Mode**
This bit enables the multi-processor communication mode. The MPCM feature is not discussed in this book.

Notice that FE, DOR, and PE are valid until the receive buffer (UDR) is read. Always set these bits to zero when writing to UCSRA.

**Figure 11-13. UCSRA: USART Control and Status Register A**

| RXCIE | TXCIE | UDRIE | RXEN | TXEN | UCSZ2 | RXB8 | TXB8 |
|---|---|---|---|---|---|---|---|

**RXCIE (Bit 7): Receive Complete Interrupt Enable**
To enable the interrupt on the RXC flag in UCSRA you should set this bit to one.

**TXCIE (Bit 6): Transmit Complete Interrupt Enable**
To enable the interrupt on the TXC flag in UCSRA you should set this bit to one.

**UDRIE (Bit 5): USART Data Register Empty Interrupt Enable**
To enable the interrupt on the UDRE flag in UCSRA you should set this bit to one.

**RXEN (Bit 4): Receive Enable**
To enable the USART receiver you should set this bit to one.

**TXEN (Bit 3): Transmit Enable**
To enable the USART transmitter you should set this bit to one.

**UCSZ2 (Bit 2): Character Size**
This bit combined with the UCSZ1:0 bits in UCSRC sets the number of data bits (character size) in a frame.

**RXB8 (Bit 1): Receive data bit 8**
This is the ninth data bit of the received character when using serial frames with nine data bits. This bit is not used in this book.

**TXB8 (Bit 0): Transmit data bit 8**
This is the ninth data bit of the transmitted character when using serial frames with nine data bits. This bit is not used in this book.

Figure 11-14. UCSRB: USART Control and Status Register B

| URSEL | UMSEL | UPM1 | UPM0 | USBS | UCSZ1 | UCSZ0 | UCPOL |
|---|---|---|---|---|---|---|---|

**URSEL (Bit 7): Register Select**
This bit selects to access either the UCSRC or the UBRRH register and will be discussed more in this section.

**UMSEL (Bit 6): USART Mode Select**
This bit selects to operate in either the asynchronous or synchronous mode of operation.

0 = Asynchronous operation
1 = Synchronous operation

**UPM1:0 (Bit 5:4): Parity Mode**
These bits disable or enable and set the type of parity generation and check.

00 = Disabled
01 = Reserved
10 = Even Parity
11 = Odd Parity

**USBS (Bit 3): Stop Bit Select**
This bit selects the number of stop bits to be transmitted.

0 = 1 bit
1 = 2 bits

**UCSZ1:0 (Bit 2:1): Character Size**
These bits combined with the UCSZ2 bit in UCSRB set the character size in a frame and will be discussed more in this section.

**UCPOL (Bit 2): Clock Polarity**
This bit is used for synchronous mode only and will not be covered in this section.

**Figure 11-15. UCSRC: USART Control and Status Register C**

**Table 11-5: Values of UCSZ2:0 for Different Character Sizes**

| UCSZ2 | UCSZ1 | UCSZ0 | Character Size |
|---|---|---|---|
| 0 | 0 | 0 | 5 |
| 0 | 0 | 1 | 6 |
| 0 | 1 | 0 | 7 |
| 0 | 1 | 1 | 8 |
| 1 | 1 | 1 | 9 |

*Note: Other values are reserved. Also notice that UCSZ0 and UCSZ1 belong to UCSRC and UCSZ2 belongs to UCSRB*

baud rate programmable

ATmega32 (8MHz/10MHz)

$$\text{desired baud rate} = \frac{f_{osc}}{16(X+1)}$$

(br)

X = value loaded into UBRR

$$X = \frac{f_{osc}}{br(16)} - 1$$

---

br = 9600

$f_{osc}$ = 8 MHz

$X = 0x33 \Rightarrow 51_{10} \Rightarrow$ UBRRL = 0x33

$$BR = \frac{f_{osc}}{(X+1)16}$$

$$X = \frac{f_{osc}}{16\,BR} - 1$$

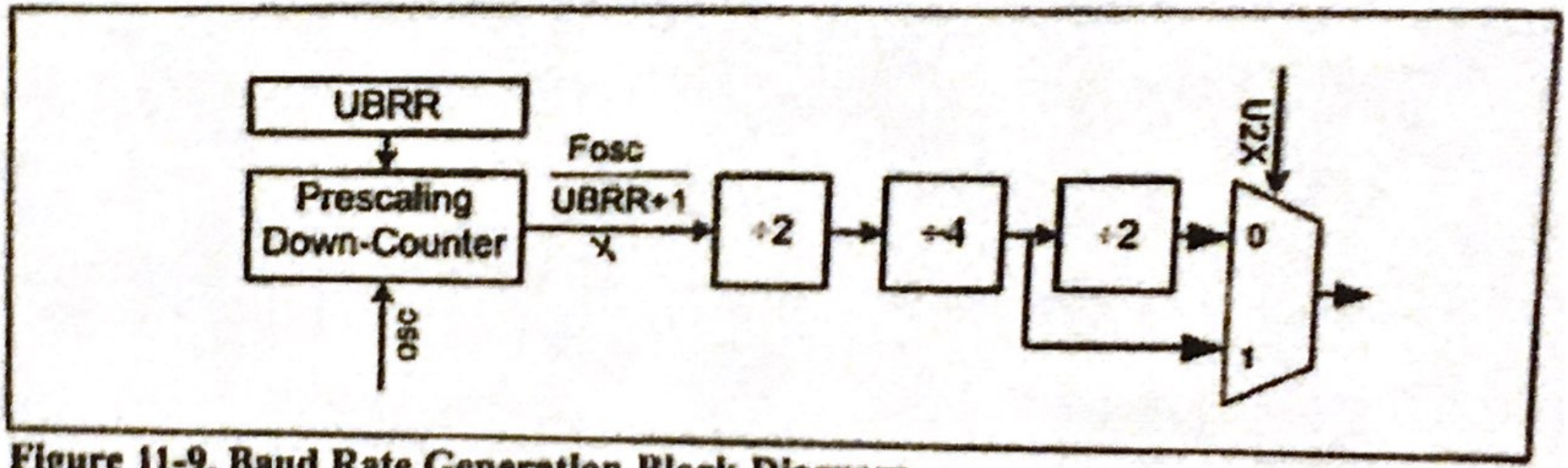
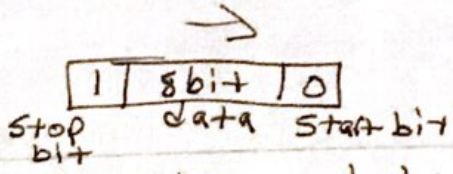



**Figure 11-9. Baud Rate Generation Block Diagram**




Write a C program to transmit 'G' continuously. Use polling method.
9600 baud. $F_{osc}$ = 8 MHz / 7.373 MHz / 11.0592 MHz

$$br = \frac{f_{osc}}{16(X+1)}$$

$$X = \frac{f_{osc}}{br(16)} - 1 \approx 51 \Rightarrow UBRRL = 0x33$$

UBRR (16 bits)
UBRRH
UBRRL

```
int main() {
    UCSRB = (1<<TXEN); // UCSZ2 = 0
                       // enable transmiss; need 8 bits data?
    UCSRC = (1<<UCSZ1) | (1<<UCSZ0)
            | (1<<URSEL);
            // URSEL ⇒ UCSRC use
            // UCSZ2: 0 ⇒ 8 bits
    UBRRL = 0x33; // baud rate 9600

    while(1) {
        while ( !(UCSRA & (1<<UDRE)));
                // wait for trans to complete
        UDR = 'G'; // write to UDR which starts transmission
    }
    return 0;
}
```